Lab Report: Basic Image Intensity Transformations

Student Name

February 2, 2026

Abstract

This report details an experiment in digital image processing where four unknown intensity transformations on a given input image were identified and replicated. The transformations were determined to be color inversion, contrast enhancement, and gamma correction (both increased and decreased). The replication was performed using Python with the Pillow (PIL) library, and the parameters were fine-tuned to match the provided output images.

1 Objective

The primary objective of this lab was to analyze a set of four output images, each derived from a single input image through an unknown transformation. The goal was to first visually identify the specific intensity transformation applied to produce each output image and then to write a program to replicate these transformations as closely as possible.

2 Methodology

The process involved two main stages: analysis and implementation.

2.1 Analysis

Visual inspection of the input and output images was the first step. By comparing the histogram and pixel intensity distribution between the input and each result, the following transformations were hypothesized:

- **Result1.png:** The colors appeared to be flipped, with dark areas becoming light and light areas becoming dark. This pointed to a **color inversion**.
- **Result2.png:** The distinction between light and dark areas was much sharper than in the original, indicating an **increase in contrast**.
- **Result3.png:** The image was significantly brighter overall, indicating a **gamma correction with gamma ¡ 1.0** (e.g., 0.4) which brightens the image.
- **Result4.png:** The image was significantly darker overall, indicating a **gamma correction with gamma ¿ 1.0** (e.g., 2.2) which darkens the image.

2.2 Implementation

To replicate these transformations, a Python script was developed using the Pillow library (a fork of PIL).

- **Color Inversion:** The `ImageOps.invert()` function was used, which performs a pixel-wise inversion of the image.
- **Contrast Adjustment:** Initially, `ImageOps.autocontrast()` was used, but it did not provide a good match. The approach was refined using the `ImageEnhance.Contrast()` class, which allows for a specific enhancement factor. A factor of 2.0 was found to be a close match.

- **Gamma Correction:** Gamma correction was implemented using a power-law transformation, defined by the equation:

$$O = c \cdot I^{\gamma}$$

Where O is the output pixel value, I is the input pixel value (normalized to [0, 1]), and γ is the gamma value. This was applied using the `Image.point()` method with a lambda function. A gamma of 0.4 was used to brighten the image (Result 3) and a gamma of 2.2 was used to darken it (Result 4).

3 Python Code

The following Python script was used to generate the final output images.

```python
from PIL import Image, ImageOps, ImageEnhance

def transform_images(input_path):
    """
    Applies various transformations to an input image and saves the results.
    """
    try:
        # Open the input image
        with Image.open(input_path) as img:
            # Ensure image is in a suitable mode (e.g., 'L' for grayscale)
            if img.mode != 'L':
                img = img.convert('L')

            # --- Result1: Color Inversion ---
            img_result1 = ImageOps.invert(img)
            img_result1.save("Result1_generated.png")

            # --- Result2: Increased Contrast (Refined) ---
            contrast_enhancer = ImageEnhance.Contrast(img)
            img_result2 = contrast_enhancer.enhance(2.0)
            img_result2.save("Result2_generated.png")

            # --- Result3: Increased Gamma (Brighter - Refined) ---
            gamma_brighter = 0.4
            img_result3 = img.point(lambda x: int(255 * ((x / 255) ** gamma_brighter)))
            img_result3.save("Result3_generated.png")

            # --- Result4: Decreased Gamma (Darker - Refined) ---
            gamma_darker = 2.2
            img_result4 = img.point(lambda x: int(255 * ((x / 255) ** gamma_darker)))
            img_result4.save("Result4_generated.png")

            print("All transformations have been applied and saved.")

    except FileNotFoundError:
        print(f"Error: The file '{input_path}' was not found.")
    except Exception as e:
        print(f"An error occurred: {e}")

if __name__ == "__main__":
    transform_images("input.png")
```

Listing 1: Python script for image transformations using Pillow.

4 Results

The generated images were compared to the provided result images. The filenames for the generated images are `ResultN_generated.png`.

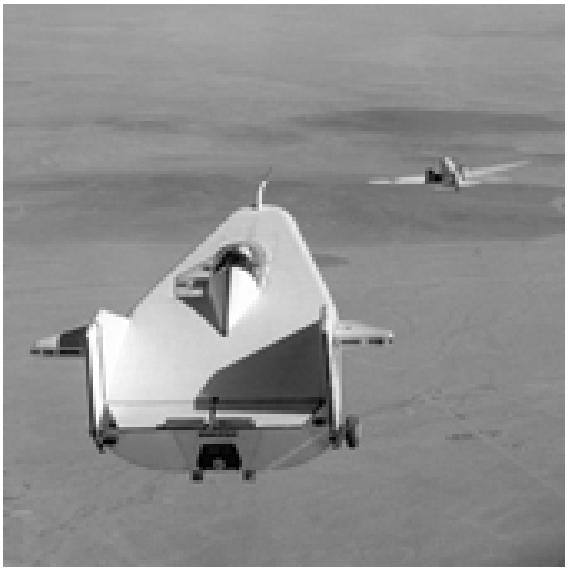

Figure 1: Original Input Image

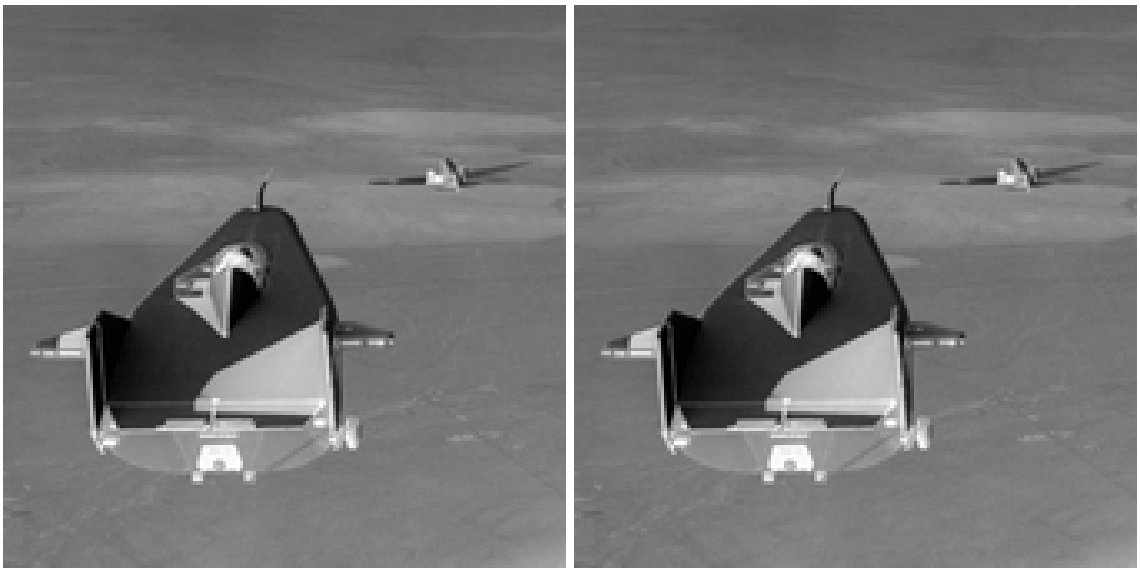

Figure 2: Result 1 (Color Inversion): Provided (left) vs. Generated (right)

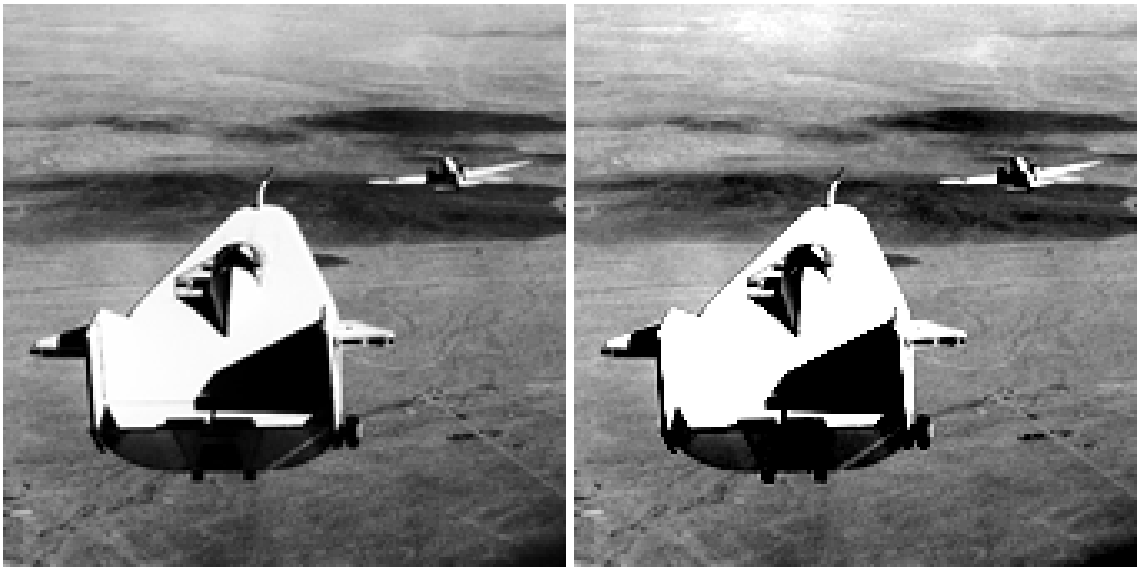

Figure 3: Result 2 (Increased Contrast): Provided (left) vs. Generated (right)

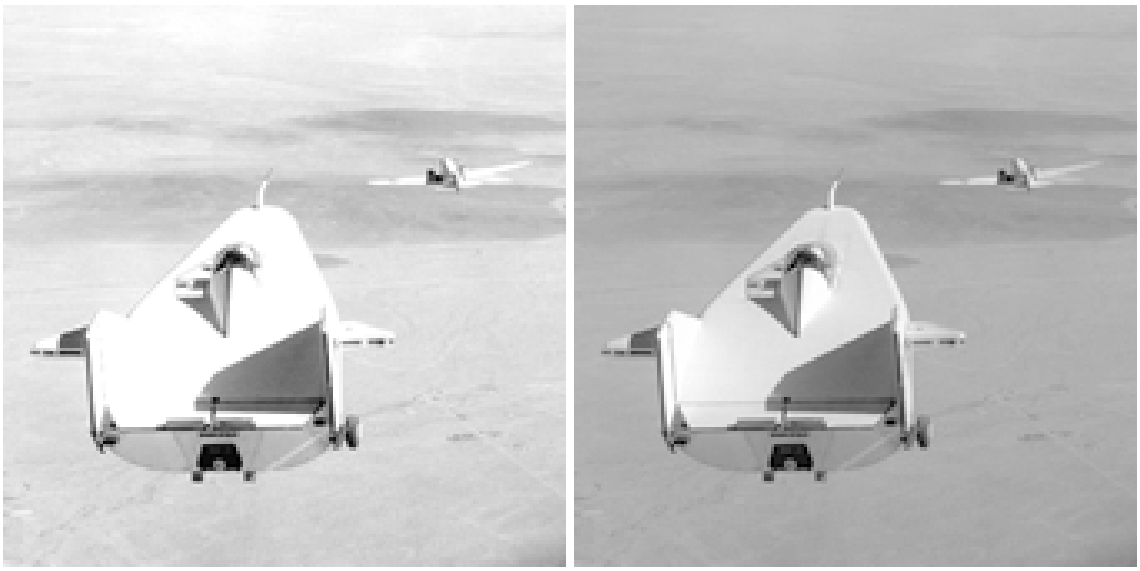

Figure 4: Result 3 (Increased Gamma): Provided (left) vs. Generated (right)

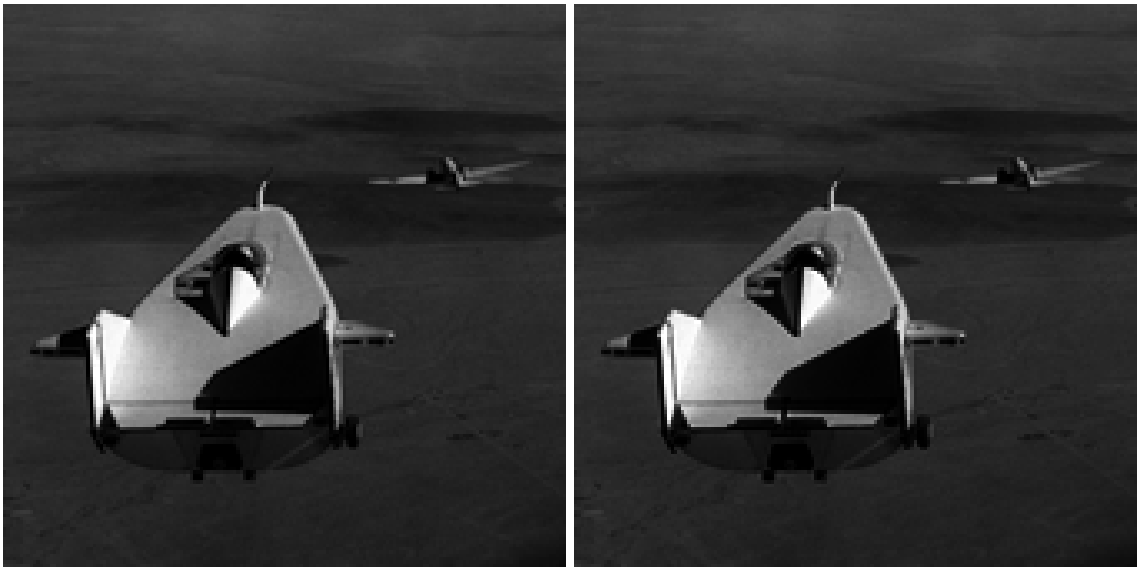

Figure 5: Result 4 (Decreased Gamma): Provided (left) vs. Generated (right)

5 Conclusion

The experiment successfully identified and replicated the four image transformations. Color inversion was replicated perfectly. Contrast and gamma correction required an iterative process of adjusting parameters to achieve a close visual match. This highlights that while the type of transformation can be identified, the exact parameters often need to be found through experimentation, especially without precise metadata. The Pillow library in Python proved to be an effective tool for performing these intensity transformations.